

# práctica 1

## SISTEMAS OPERATIVOS

Lara Berenguer De Casas

# ÍNDICE

## Contenido

|                                                            |    |
|------------------------------------------------------------|----|
| LINK GITHUB.....                                           | 4  |
| EJERCICIO 1: APARTADO A .....                              | 5  |
| 1.1    ENUNCIADO DEL EJERCICIO .....                       | 5  |
| 1.1.1 ESTRUCTURA DEL ÁRBOL DE PROCESOS.....                | 5  |
| 1.1.2 RESULTADO ESPERADO.....                              | 5  |
| 1.2    IDENTIFICACIÓN Y JUSTIFICACIÓN DE TAREAS .....      | 6  |
| 1.2.1 RECEPCIÓN Y VALIDACIÓN DE ARGUMENTOS.....            | 6  |
| 1.2.2 GENERACIÓN DE LA JERARQUÍA DE PROCESOS .....         | 6  |
| 1.2.3 CONSTRUCCIÓN DE LAS COLUMNAS.....                    | 6  |
| 1.2.4 MANTENIMIENTO Y COMPROBACIÓN DE LA ESTRUCTURA.....   | 7  |
| 1.3    DIAGRAMA DE SECUENCIA .....                         | 7  |
| 1.4    ANÁLISIS DETALLADO DEL CÓDIGO FUENTE.....           | 8  |
| 1.4.1 FUNCIÓN VALIDAR_ARGUMENTOS.....                      | 8  |
| 1.4.2 FUNCION crear_primera_fila().....                    | 9  |
| 1.4.3 FUNCION crear_columna.....                           | 10 |
| 1.4.4 MANTENIMIENTO DE LA ESTRUCTURA PARA PSTREE -C.....   | 10 |
| 1.4.5 ESPERA DEL PROCESO PRINCIPAL, FUNCIÓN MAIN().....    | 11 |
| 1.4.6 COMPROBACIÓN DE LA ESTRUCTURA CON PSTREE -C.....     | 11 |
| EJERCICIO 1: APARTADO B .....                              | 12 |
| 1.1    ENUNCIADO DEL EJERCICIO .....                       | 12 |
| 1.1.1 ESTRUCTURA DEL ÁRBOL DE PROCESOS.....                | 12 |
| 1.1.2 RESULTADO ESPERADO.....                              | 13 |
| 1.2    IDENTIFICACIÓN Y JUSTIFICACIÓN DE TAREAS .....      | 13 |
| 1.2.1 RECEPCIÓN Y VALIDACIÓN DE ARGUMENTOS.....            | 14 |
| 1.2.2 GENERACIÓN DE LA JERARQUÍA DE PROCESOS.....          | 14 |
| 1.2.3 TEMPORIZACIÓN SIN BLOQUEO.....                       | 14 |
| 1.2.4 EJECUCIÓN DE LA ORDEN MEDIANTE SEÑALES.....          | 14 |
| 1.2.5 COMPORTAMIENTO REACTIVO Y EJECUCIÓN DE COMANDOS..... | 15 |

|                                                               |    |
|---------------------------------------------------------------|----|
| 1.2.6 NOTIFICACIÓN Y TRANSICIÓN A LA FASE DE DESTRUCCIÓN..... | 15 |
| 1.2.7 DESTRUCCIÓN CONTROLADA DEL ÁRBOL.....                   | 15 |
| 1.3 DIAGRAMA DE SECUENCIA .....                               | 16 |
| 1.4 ANÁLISIS DETALLADO DEL CÓDIGO FUENTE.....                 | 17 |
| 1.4.1 FUNCIONES GLOBALES Y DE UTILIDAD.....                   | 17 |
| 1.4.2 PROCESO INICIAL EXEC Y CREACIÓN DEL PROCESO a.....      | 18 |
| 1.4.3 PROCESO A Y CREACIÓN DEL PROCESO B.....                 | 18 |
| 1.4.4 PROCESO B Y LOS PROCESOS X, Y, Z.....                   | 19 |
| 1.4.5 PROCESO Z Y LA TEMPORIZACIÓN.....                       | 20 |
| 1.4.6 EJECUCIÓN DE PSTree.....                                | 21 |
| 1.4.7 DESTRUCCIÓN CONTROLADA DEL ÁRBOL.....                   | 21 |
| EJERCICIO 2.....                                              | 22 |
| 2.1 ENUNCIADO DEL EJERCICIO .....                             | 22 |
| 2.1.1 ESTRUCTURA DE PROCESOS.....                             | 23 |
| 2.1.2 RESULTADO ESPERADO.....                                 | 23 |
| 2.2 IDENTIFICACIÓN Y JUSTIFICACIÓN DE TAREAS.....             | 23 |
| 2.2.1 RECEPCIÓN DE ARGUMENTOS Y PREPARACIÓN DEL ARCHIVO ..... | 24 |
| 2.2.2 CREACIÓN DE PROCESOS Y TUBERÍAS .....                   | 24 |
| 2.2.3 COMUNICACIÓN PADRE E HIJO Y CREACIÓN DE FRAGMENTOS..... | 24 |
| 2.2.4 SINCRONIZACIÓN Y FINALIZACIÓN.....                      | 24 |
| 2.3 DIAGRAMA DE SECUENCIA .....                               | 25 |
| 2.4 ANÁLISIS DETALLADO DEL CÓDIGO FUENTE.....                 | 26 |
| 2.4.1 FUNCION VALIDAR_argumentos().....                       | 26 |
| 2.4.2 FUNCION main().....                                     | 27 |
| 2.4.3 FUNCION PROCESAR_fragmento().....                       | 28 |
| 2.4.4 FUNCION PROCESO_HIJO().....                             | 29 |
| 2.4.5 EJECUCIÓN.....                                          | 30 |
| EJERCICIO 3.....                                              | 31 |
| 3.1 ENUNCIADO DEL EJERCICIO .....                             | 31 |
| 3.1.1 ESTRUCTURA DEL ÁRBOL DE PROCESOS.....                   | 31 |
| 3.1.2 RESULTADO ESPERADO.....                                 | 31 |
| 3.2 IDENTIFICACIÓN Y JUSTIFICACIÓN DE TAREAS.....             | 32 |
| 3.2.1 RECEPCIÓN Y VALIDACIÓN DE ARGUMENTOS.....               | 32 |

|       |                                                    |           |
|-------|----------------------------------------------------|-----------|
| 3.2.2 | creación de la cadena de x procesos.....           | 32        |
| 3.2.3 | creación de los y procesos finales.....            | 32        |
| 3.2.4 | comunicación mediante memoria compartida .....     | 33        |
| 3.2.5 | sincronización y finalización de los procesos..... | 33        |
| 3.3   | DIAGRAMA DE SECUENCIA .....                        | <b>33</b> |
| 3.4   | ANÁLISIS DETALLADO DEL CÓDIGO FUENTE.....          | <b>35</b> |
| 3.4.1 | función validar_argumentos().....                  | 35        |
| 3.4.2 | función main().....                                | 35        |
| 3.4.3 | función crear_cadena().....                        | 36        |
| 3.4.4 | función crear_hijos_finales().....                 | 37        |
| 3.4.5 | función mostrar_superpadre().....                  | 38        |
| 3.4.6 | ejecución.....                                     | 38        |

## LINK GITHUB

<https://github.com/laraaberenguer/S0>

# EJERCICIO 1: APARTADO A

## 1.1 ENUNCIADO DEL EJERCICIO

Realiza un programa llamado `mallla.c` que produzca el siguiente árbol de procesos. El programa recibirá dos argumentos 'x' e 'y' que representan el número de filas y columnas. Para comprobar la estructura de procesos que realmente estamos creando debemos emplear la llamada al sistema "pstree -c"

### 1.1.1 ESTRUCTURA DEL ÁRBOL DE PROCESOS

La jerarquía de procesos que se debe crear es la siguiente:

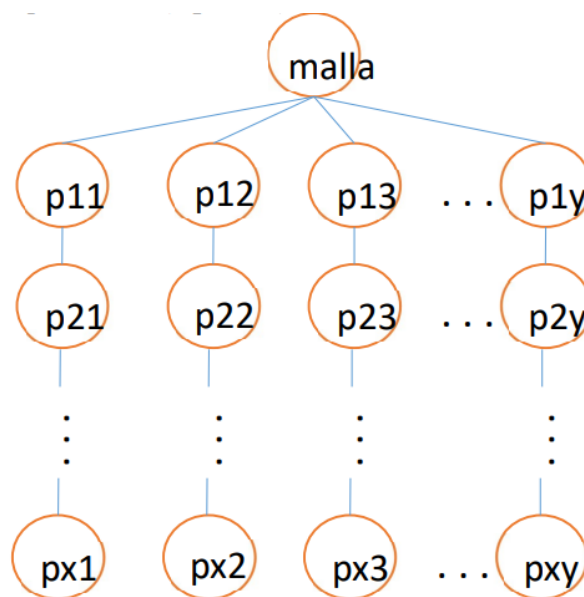


Figura 1.1: Diagrama del árbol de procesos a generar.

### 1.1.2 RESULTADO ESPERADO

Por ejemplo para la ejecución:

```
./malla 3 4
```

Se generarán 3 filas (x=3) y 4 columnas (y=4) de procesos, de manera que existirán 12 procesos pertenecientes a la malla, además del proceso principal malla.

## 1.2 IDENTIFICACIÓN Y JUSTIFICACIÓN DE TAREAS

Para comenzar con el ejercicio, primero tenemos que identificar las diferentes tareas a realizar para obtener la estructura de procesos deseada con su jerarquía correspondiente.

A partir de los valores 'x' e 'y' que recibe el programa, se ha de crear una estructura de 'x' filas e 'y' columnas, el proceso inicial 'malla' actúa como raíz del árbol y a partir de él se construyen las distintas ramas.

### 1.2.1 RECEPCIÓN Y VALIDACIÓN DE ARGUMENTOS

El programa necesita recibir dos valores que van a ser los que determinaran el tamaño del árbol.

El primer argumento, x, representa el número de filas que tendrá cada rama de la malla y, por otro lado, el segundo argumento, y, será el número de columnas que nacen del proceso principal.

Primero habrá que comprobar que se han introducido ambos valores para que el programa pueda realizar su función, además también habrá que comprobar que las dimensiones sean válidas, deberán de ser mayores que cero.

### 1.2.2 GENERACIÓN DE LA JERARQUÍA DE PROCESOS

Según la estructura a seguir del enunciado, podemos observar que los únicos hijos directos de 'malla' son los procesos de la primera fila: p11, p12, p13...p1y.

De manera que, el argumento que determina el número de hijos directos del proceso principal es 'y' (número de columnas).

Cada proceso forma una rama diferente del árbol y es responsable de continuar con la construcción de su propia columna (hacia abajo).

Es importante recalcar que el resto de los procesos que no son la primera fila no son hijos directos del proceso principal, sino hijos de los hijos de malla y así sucesivamente.

### 1.2.3 CONSTRUCCIÓN DE LAS COLUMNAS

Como se ha comentado en el apartado anterior, cada proceso de la primera fila debe continuar con su propia columna, por lo que cada proceso debe ser padre del proceso situado justo debajo de él. Por ejemplo, en la columna 1: p11 -> p21 -> p31 -> ... -> px1.

Para generar estas relaciones utilizaremos la llamada al sistema `fork()`, que lo que hará será que un proceso cree un nuevo proceso hijo. Es muy importante que tras realizar esta llamada diferenciamos entre el proceso padre y el hijo, para que solo el hijo continúe construyendo y así cumplir con la estructura.

## 1.2.4 MANTENIMIENTO Y COMPROBACIÓN DE LA ESTRUCTURA

Una vez tenemos el árbol completo es necesario mantener los procesos activos durante el tiempo suficiente para poder comprobar la estructura. Si los procesos finalizaran inmediatamente después de la creación, algunas ramas podrían desaparecer antes de la comprobación.

Para solucionar esto, en los procesos finales de cada columna utilizamos la llamada `pause()`, que detiene su ejecución hasta la recepción de una señal. Al mismo tiempo los procesos de niveles superiores permanecen esperando la finalización de sus hijos con `wait()`.

Haremos uso del comando `ps tree -c`, como bien indica el enunciado, para comprobar que el proceso 'malla' cuenta con 'y' ramas y que cada una tiene 'x' procesos.

## 1.3 DIAGRAMA DE SECUENCIA

El siguiente diagrama muestra la secuencia de creación de los procesos que forman la malla y representa las relaciones y dependencias entre los procesos, se han tomado como ejemplo los argumentos  $x=3$  filas e  $y=2$  columnas.



También se comprueba que las dimensiones de los argumentos sean válidas, que sean mayores que cero. Si en alguna de las dos comprobaciones que realizamos a los argumentos no se cumple lo deseado, el programa muestra un mensaje de error y se finaliza mediante la función `exit()`.

Llamadas al sistema utilizadas: `exit()`.

```
void validar_argumentos(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Error: No se han introducido los argumentos correctamente\n");
        exit(1);
    }

    if (atoi(argv[1]) <= 0 || atoi(argv[2]) <= 0) {
        printf("Error: No se han introducido los argumentos correctamente, deben de ser mayores que cero\n");
        exit(1);
    }
}
```

## 1.4.2 FUNCION `crear_primera_fila()`

Con los argumentos validados, comienza la construcción de la malla. El proceso principal hace uso de un bucle que se repite tantas veces como columnas tenga la estructura (argumento `y`).

En cada iteración se realiza una llamada `fork()`. De esta manera, ‘malla’ crea un proceso hijo que corresponde a cada elemento de la primera fila: `p11`, `p12`, ..., `p1y`.

El valor que devuelve `fork()` nos sirve para saber que proceso va a continuar ejecutándose. Si devuelve `-1` se ha producido un error, si devuelve `0` el proceso actual es el hijo y, finalmente, si devuelve un valor positivo, el proceso que continua es el padre.

Cuando estamos situados en el proceso hijo, este no crea nuevas columnas, sino que se centra en la construcción de su propia columna. En cambio, el proceso ‘malla’ continua el bucle para generar la siguiente columna nueva.

Llamadas al sistema utilizadas: `fork()`, `getpid()`, `getppid()`, `exit()`.

```
void crear_primera_fila(int x, int y) {
    int j;
    pid_t pid;

    for (j = 1; j <= y; j++) {
        pid = fork();

        if (pid < 0) {
            perror("Error en fork");
            exit(1);
        }

        if (pid == 0) {
            printf("Soy p%d%d, mi PID es %d y mi padre es %d\n", 1, j, getpid(), getppid());

            crear_columna(x, j);

            exit(0);
        }
    }
}
```

### 1.4.3 FUNCION crear\_COLUMNa

Una vez creado el proceso de la primera fila, este debe de crear el resto de los procesos de su propia columna. Para ello utilizamos un segundo bucle, que comenzará en la segunda fila, ya que el proceso de la primera ya está creado.

En cada iteración se realiza una llamada fork(). El proceso hijo es ahora el responsable de continuar el bucle.

El proceso padre no debe de continuar generando nuevos procesos, porque si no la estructura sería incorrecta, tanto padre como hijo generarían procesos y no se obtendría la cadena vertical que se pide en el ejercicio.

Para cumplir con esto el padre se queda esperando con wait(NULL) a que termine el hijo y luego termina su ejecución, el hijo es el único que continua con la siguiente iteración.

Llamadas al sistema utilizadas: fork(), getpid(), getppid(), exit(), wait().

```
void crear_columna(int x, int j) {
    int i;
    pid_t pid;

    for (i = 2; i <= x; i++) {
        pid = fork();

        if (pid < 0) {
            perror("Error en fork");
            exit(1);
        }

        if (pid > 0) {
            wait(NULL);
            exit(0);
        }

        printf("Soy p%d, mi PID es %d y mi padre es %d\n", i, j, getpid(), getppid());
    }

    pause();
}
```

### 1.4.4 MANTENIMIENTO DE LA ESTRUCTURA PARA PSTREE -C

En la ultima fila ya no es necesario generar mas procesos. Hemos de tener en cuenta que si el último proceso finalizara inmediatamente los procesos que están esperando irían terminando también y la estructura desaparecería.

Para ello utilizamos la llamada al sistema pause() en el último proceso de cada columna para que este permanezca bloqueado hasta que reciba una señal. Así conservamos la estructura y se puede ejecutar el comando pstree -c.

Llamadas al sistema utilizadas: pause().

```
}  
    pause();  
}
```

## 1.4.5 ESPERA DEL PROCESO PRINCIPAL, FUNCIÓN main()

La función main() coordina la ejecución del programa. Primero llama a validar\_argumentos() y convierte estos a enteros mediante atoi(). A continuación, muestra el PID del proceso principal y llama a crear\_primera\_fila() para comenzar la construcción de la malla.

Después de generar los procesos de la primera fila, el proceso principal no debe de generar nuevos procesos.

Como 'malla' tiene 'y' hijos directos realiza 'y' llamadas a wait(NULL). Así se mantiene activo mientras las ramas de la malla se construyen.

Cuando ya han terminado todos los procesos de la primera fila, significando así que también han finalizado sus respectivas columnas, el proceso principal finaliza mediante exit(0).

Llamadas al sistema utilizadas: wait(), exit(), getpid().

```
int main(int argc, char *argv[]) {  
    int x, y, j;  
  
    validar_argumentos(argc, argv);  
  
    x = atoi(argv[1]);  
    y = atoi(argv[2]);  
  
    printf("Soy malla, mi PID es %d\n", getpid());  
  
    crear_primera_fila(x, y);  
  
    for (j = 1; j <= y; j++) {  
        wait(NULL);  
    }  
  
    printf("Soy malla (%d) y termino\n", getpid());  
  
    exit(0);  
}
```

## 1.4.6 COMPROBACIÓN DE LA ESTRUCTURA CON PSTREE -C

Ahora comprobamos en la terminal la estructura de procesos mediante 'pstree -c'. A continuación, adjunto las capturas de la terminal al ejecutar el programa y la llamada al sistema. Gracias a la llamada pause() podemos realizar la

comprobación en la terminal, ya que los procesos de la última fila permanecen bloqueados. En este caso se ha hecho la ejecución con los siguientes argumentos de ejemplo: x=2 filas, y =3 columnas.

```
alumno@alumno-VirtualBox:~/S0/Practical/Ejercicio1/ApartadoA$ ./malla 2 3
Soy malla, mi PID es 5526
Soy p11, mi PID es 5527 y mi padre es 5526
Soy p12, mi PID es 5528 y mi padre es 5526
Soy p21, mi PID es 5530 y mi padre es 5527
Soy p13, mi PID es 5529 y mi padre es 5526
Soy p23, mi PID es 5532 y mi padre es 5529
Soy p22, mi PID es 5531 y mi padre es 5528
```

```
xfce4-terminal-bash
├── malla
│   ├── malla
│   ├── malla
│   └── malla
└── pstree
```

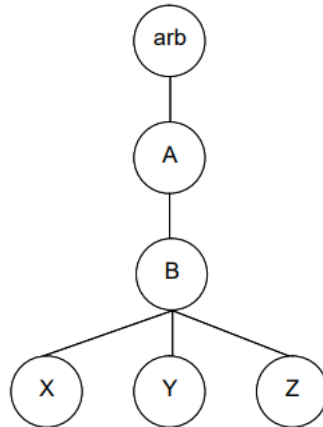
## EJERCICIO 1: APARTADO B

### 1.1 ENUNCIADO DEL EJERCICIO

Realiza un programa llamado ejec.c que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando “pstree”. El proceso Z no puede utilizar el comando “sleep”, por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento. Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).

#### 1.1.1 ESTRUCTURA DEL ÁRBOL DE PROCESOS

La jerarquía de procesos que se debe crear es la siguiente:



## 1.1.2 RESULTADO ESPERADO

A continuación se muestra un ejemplo de la salida esperada por la terminal:

El programa debe mostrar por la salida estándar la siguiente información:

```

$ ejec 15
Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
/*Tras un intervalo de 15 segundos aparecerá*/
/*Resultado del comando "pstree" */
Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero
  
```

## 1.2 IDENTIFICACIÓN Y JUSTIFICACIÓN DE TAREAS

Para implementar el programa tenemos que saber distinguir entre diversas tareas que permiten construir el árbol con la estructura deseada, realizar la comunicación entre los procesos mediante señales y asegurarnos de la correcta finalización del árbol.

## 1.2.1 RECEPCIÓN Y VALIDACIÓN DE ARGUMENTOS

El programa ejec recibe mediante la línea de comandos un único argumento que indica el número de segundos que deben transcurrir antes de que el proceso Z envíe una señal al proceso A. De manera que lo que debemos hacer es comprobar que se haya recibido este argumento y que sea válido para usarse como tiempo.

El valor recibido se convierte en un valor numérico que usaremos en la llamada alarm(). Este tiempo será el que empleará el proceso Z para programar una alarma.

## 1.2.2 GENERACIÓN DE LA JERARQUÍA DE PROCESOS

El programa debe generar una estructura de procesos que comienza con el proceso inicial 'ejec', su hijo A, el hijo B de A y, finalmente, los tres hijos X,Y y Z creados por B.

Para construir esta estructura se utiliza la llamada al sistema fork(). Cada proceso encargado de crear sus hijos tendrá que realizar las llamadas necesarias a fork(), posteriormente habrá que distinguir el proceso padre y el proceso hijo para saber cómo continuar.

## 1.2.3 TEMPORIZACIÓN SIN BLOQUEO

El enunciado dice que el proceso Z debe esperar el número de segundos que se ha indicado mediante el argumento, sin utilizar la llamada sleep(). Para esto tendremos que hacer uso de la llamada al sistema alarm(). El proceso Z programa una alarma mediante alarm(segundos). Después de que pase el tiempo el sistema envía al proceso Z la señal SIGALRM. Previamente, Z habrá asociado a esta señal la función manejadora mediante signal(). Mientras espera la llegada de la señal Z permanecerá detenido mediante pause().

## 1.2.4 EJECUCIÓN DE LA ORDEN MEDIANTE SEÑALES

El proceso A debe ejecutar el comando 'pstree' cuando reciba la señal enviada por Z. Para ello, el proceso A programa previamente un manejador mediante la llamada signal(), así queda preparado para reaccionar cuando reciba la señal.

Cuando vence la alarma programada por Z, se ejecuta su manejador asociado a SIGALRM. Desde el manejador Z envía una señal al proceso A mediante la llamada kill().

Al recibir la señal, A ejecuta el manejador correspondiente, desde el cual se realiza la ejecución del comando pstree. De esta forma el comando se ejecuta después de que haya transcurrido el tiempo indicado como argumento por el usuario.

## 1.2.5 COMPORTAMIENTO REACTIVO Y EJECUCIÓN DE COMANDOS

Una vez creado el árbol, los procesos A,B,X e Y deben permanecer a la espera mientras el proceso Z realiza la temporización mediante alarm(). Para poder realizar esa acción utilizamos la llamada pause(), que mantiene detenido el proceso hasta que recibe una señal.

Cuando Z recibe SIGALRM, envía una señal a A mediante kill(). Al recibirla, A ejecuta el manejador que había asociado previamente a esa señal y entonces inicia la ejecución del comando pstree.

Para ejecutar pstree sin sustituir la ejecución del proceso A necesitamos crear un proceso auxiliar mediante fork(). Este proceso es el que ejecutará el comando mientras A podrá continuar después de la ejecución con el control y la destrucción deseada del árbol.

## 1.2.6 NOTIFICACIÓN Y TRANSICIÓN A LA FASE DE DESTRUCCIÓN

Después de ejecutar el comando pstree, debe comenzar la fase de destrucción del árbol de procesos. Los procesos que permanecían detenidos mediante pause() son avisados mediante señales para que puedan continuar y así finalizar. La destrucción no comenzará hasta que se haya realizado la temporización de Z, se haya enviado la señal a A y se haya ejecutado el comando pstree. Al utilizar señales se permite coordinar el paso de la fase de espera a la de destrucción para que así los procesos no finalicen antes de que se haya mostrado la estructura completa.

## 1.2.7 DESTRUCCIÓN CONTROLADA DEL ÁRBOL

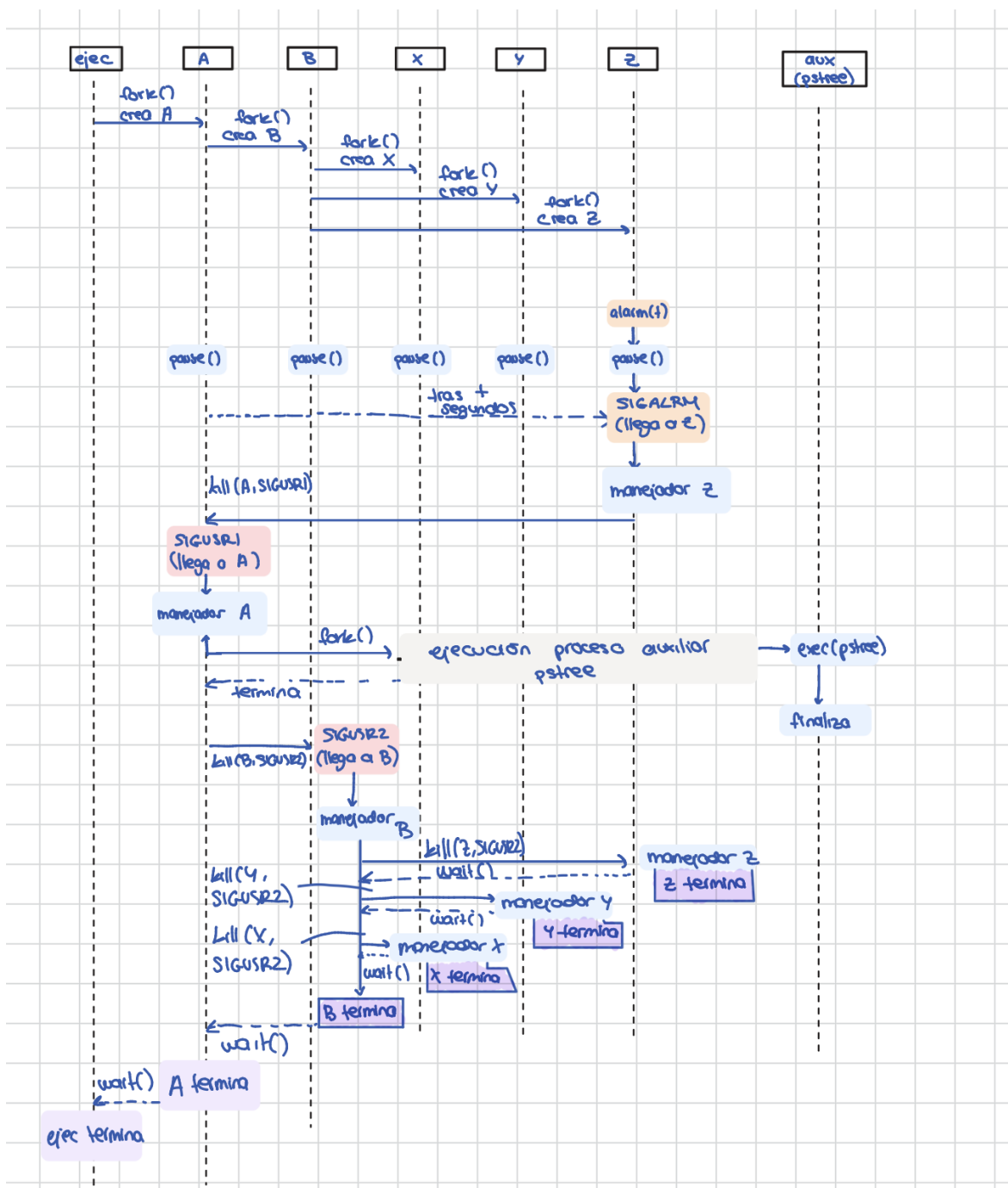
Es importante que ningún proceso padre finalice antes que sus hijos, para ello utilizamos la llamada al sistema wait(). Así cada proceso padre espera a que sus procesos hijos finalicen antes de terminar su propia ejecución.

B deberá de esperar a que finalicen X,Y y Z antes de finalizar. Después A esperará a que finalice B y, finalmente, el proceso inicial 'ejec' esperará a la finalización de A.

De esta manera se cumple lo que establece el enunciado y se realiza una destrucción ordenada del árbol de procesos.

### 1.3 Diagrama De secuencia

El siguiente diagrama muestra el funcionamiento del programa, como se crean los procesos, como se realiza la temporización de Z hasta que llega la señal a A, se ejecuta pstree y finalmente se destruye el árbol de forma ordenada.



La zona de abajo del diagrama se ve algo “apretada”, por ello voy a realizar una aclaración sobre la última parte del programa. Tras la ejecución del proceso auxiliar, A le envía una señal SIGUSR2 a B que va a servir para llevar a cabo la correcta destrucción del árbol, de esta manera B enviará esta misma señal a cada uno de sus hijos (Z -> Y->X) para avisarles de que pueden finalizar (en ese orden concreto), cada uno de los hijos sale de la pausa y van terminando. Los wait() se deben a que el proceso B debe esperar a que todos sus hijos finalicen, después B terminará, a continuación, A y, finalmente, ejec.

## 1.4 ANÁLISIS DETALLADO DEL CÓDIGO FUENTE

Procedemos a realizar un análisis detallado del código que hace posible el programa, recorriendo todas las funciones y llamadas al sistema que se realizan.

### 1.4.1 FUNCIONES GLOBALES Y DE UTILIDAD

Primero se define la función manejador() que va a ser utilizada como manejador para las distintas señales que se dan en nuestro programa. Esta función está vacía ya que no necesita realizar ninguna operación, su finalidad se basa en que un proceso que se encuentra detenido por pause() continúe con su ejecución tras recibir la señal correspondiente.

También se define la función validar\_argumentos() que comprueba que el programa reciba exactamente un argumento. Luego utiliza atoi() para convertirlo a un valor entero y comprueba que sea un valor mayor a cero. Si alguna de las comprobaciones no se cumple, se muestra un mensaje de error y se finaliza mediante exit(). Si todo es correcto la función devuelve el número de segundos.

Llamadas al sistema utilizadas: exit().

```
void manejador(int signal){
}

int validar_argumentos(int argc, char *argv[]){
    int segundos;

    if (argc != 2){
        printf("Error: se ha de introducir un único argumento");
        exit(1);
    }

    segundos = atoi(argv[1]);

    if (segundos <= 0){
        printf("El numero de segundos debe ser mayor que 0\n");
        exit(1);
    }

    return segundos;
}
```

## 1.4.2 PROCESO INICIAL ejec Y CREACIÓN DEL PROCESO A

La función `main()` se encarga de coordinar el inicio y la finalización del programa. Primero llama a `validar_argumentos()` y tras validar los argumentos comenzamos con el proceso inicial `ejec`, guardaremos su pid mediante la llamada `getpid()` en la variable `pidEjec`, ya que nos hará falta para mostrar la relación con el resto de procesos en los mensajes.

El proceso `ejec` crea el proceso A mediante la llamada al sistema `fork()`. Si la llamada falla se muestra un mensaje de error y se finaliza. Si el valor devuelto por el `fork()` es 0 significa que estamos en el proceso hijo A, que llama a `proceso_A()`, de lo contrario el proceso que continua es `ejec` y el valor devuelto por `fork()` corresponde al PID de A.

El proceso `ejec` utiliza la llamada `wait(NULL)` para permanecer en espera hasta que termine su hijo A, ya que el proceso padre no puede finalizar antes que su hijo, como bien indica el enunciado.

Llamadas al sistema utilizadas: `fork()`, `getpid()`, `wait()`, `exit()`.

```
int main(int argc, char *argv[]){
    pid_t pidEjec, pidA;
    int segundos;

    segundos = validar_argumentos(argc, argv);

    pidEjec = getpid();

    printf("Soy el proceso ejec: mi pid es %d\n", getpid());

    pidA = fork();

    if (pidA == -1){
        perror("Error en fork");
        exit(1);
    }

    if (pidA == 0){
        proceso_A(pidEjec, segundos);
    }
    else{
        wait(NULL);
        printf("Soy ejec (%d) y muero\n", getpid());
    }

    return 0;
}
```

## 1.4.3 PROCESO A Y CREACIÓN DEL PROCESO B

Una vez creado el proceso A, este guarda su propio identificador mediante `getpid()` en la variable `pidA`, ya que este valor será necesario después para mostrar las relaciones en los mensajes y para que el proceso Z envíe la señal a A.

Mediante la llamada al sistema `signal()` se configura la recepción de la señal `SIGUSR1` asociando la llamada a la función `manejador()`. Esta será la señal que enviará Z a A transcurridos los segundos correspondientes.

Cuando ya hemos preparado la recepción de la señal, A crea al proceso B mediante `fork()`, realizando las mismas comprobaciones con las estructuras `'if'`, `'else if'` y `'else'` que en la creación anterior. Si estamos en el proceso hijo se llama a `proceso_B()` para continuar construyendo el árbol, mientras tanto A permanece esperando la señal correspondiente.

Llamadas al sistema utilizadas: `fork()`, `getpid()`, `getppid()`, `signal()`, `exit()`, `pause()`.

```
void proceso_A(pid_t pidEjec, int segundos){
    pid_t pidA, pidB, pidAux;

    pidA = getpid();

    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n",getpid(), getppid());

    signal(SIGUSR1, manejador);

    pidB = fork();

    if (pidB == -1){
        perror("Error en fork");
        exit(1);
    }

    if (pidB == 0){
        proceso_B(pidA, pidEjec, segundos);
    }
    else{
        pause();
    }
}
```

## 1.4.4 PROCESO B Y LOS PROCESOS X, Y, Z

Al crearse el proceso B continuamos con la configuración de la recepción de la señal `SIGUSR2`, que será utilizada posteriormente para la correcta destrucción del árbol.

B crea los procesos X,Y y Z mediante tres llamadas a `fork()`. Con las estructuras de código que llevamos usando para las anteriores creaciones de procesos nos aseguramos de que sea B únicamente el proceso que siga creando otros procesos, para evitar así que los procesos hijos creen otros descendientes. Mediante el valor que devuelve la llamada al sistema `fork()` podemos distinguir los procesos para conseguir la jerarquía deseada. Cuando se crea cada hijo, este llama a su función correspondiente: `proceso_X()`, `proceso_Y()`, `proceso_Z()`.

En los procesos X e Y se configura la recepción de la señal `SIGUSR2` mediante `signal()`, después ejecutarán `pause()` y permanecerán detenidos hasta que tras la ejecución de la temporización B envíe la señal que hace que se ejecute el manejador y cada uno de estos procesos finalice.

Llamadas al sistema utilizadas: `fork()`, `getpid()`, `getppid()`, `signal()`, `exit()`, `pause()`.

```

void proceso_B(pid_t pidA, pid_t pidEjec, int segundos){
    pid_t pidX, pidY, pidZ;

    printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi abuelo es %d\n", getpid(), getppid(), pidEjec);

    signal(SIGUSR2, manejador);

    pidX = fork();

    if (pidX == -1){
        perror("Error en fork");
        exit(1);
    }

    if (pidX == 0){
        proceso_X(pidA, pidEjec);
    }
    else {
        pidY = fork();

        if (pidY == -1){
            perror("Error en fork");
            exit(1);
        }

        if (pidY == 0){
            proceso_Y(pidA, pidEjec);
        }
    }
}

```

```

void proceso_X(pid_t pidA, pid_t pidEjec){
    printf("Soy el proceso X: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n", getpid(), getppid(), pidA, pidEjec);

    signal(SIGUSR2, manejador);
    pause();

    printf("Soy X (%d) y muero\n", getpid());

    exit(0);
}

void proceso_Y(pid_t pidA, pid_t pidEjec){
    printf("Soy el proceso Y: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo es %d\n", getpid(), getppid(), pidA, pidEjec);

    signal(SIGUSR2, manejador);
    pause();

    printf("Soy Y (%d) y muero\n", getpid());

    exit(0);
}

```

## 1.4.5 PROCESO Z Y LA TEMPORIZACIÓN

El proceso Z, en la función `proceso_Z()`, configura dos señales distintas mediante `signal()`, por un lado prepara la recepción de `SIGALRM` que será la señal que reciba cuando finalice la alarma programada. Por otro lado, también prepara la recepción de `SIGUSR2` que será utilizada por B para la destrucción del árbol.

El proceso Z programa la alarma mediante `alarm(segundos)` utilizando el argumento recibido para los segundos, tras ella se ejecuta `pause()` y el proceso permanece detenido hasta recibir `SIGALRM`. Mediante la llamada al sistema `kill()` se le envía la señal `SIGUSR1` al proceso A para que este ejecute el comando 'pstree'.

Tras avisar a A, Z vuelve a ejecutar `pause()` hasta recibir la señal que le envía B (`SIGUSR2`) que le permite finalizar su ejecución.

Llamadas al sistema utilizadas: getpid(),getppid(), signal(), exit(), pause(),alarm(), kill()).

```
void proceso_Z(pid_t pidA, pid_t pidEjec, int segundos){
    printf("Soy el proceso Z: mi pid es %d. Mi padre es %d. Mi abuelo es %d.Mi bisabuelo es %d\n", getpid(), getppid(), pidA, pidEjec);

    signal(SIGALRM, manejador);
    signal(SIGUSR2, manejador);

    alarm(segundos);
    pause();

    kill(pidA, SIGUSR1);
    pause();

    printf("Soy Z (%d) y muero\n", getpid());

    exit(0);
}
```

## 1.4.6 EJECUCIÓN DE PSTREE

El proceso A, dentro de la función proceso\_A(), permanece detenido mediante pause() hasta recibir SIGUSR1 de Z. Una vez recibida la señal se crea el proceso auxiliar con fork(), este proceso ejecutará el comando ‘pstree’ utilizando execlp().

A espera mediante wait() a que el proceso auxiliar termine. Así ‘pstree’ puede mostrar el árbol sin sustituir al propio proceso A.

Llamadas al sistema utilizadas: fork(), pause(), wait(), exit(), execlp().

```
pidAux = fork();

if (pidAux == -1){
    perror("Error en fork");
    exit(1);
}

if (pidAux == 0){
    execlp("pstree", "pstree", NULL);
    perror("Error en exec");
    exit(1);
}
else{
    wait(NULL);

    kill(pidB, SIGUSR2);
    wait(NULL);

    printf("Soy A (%d) y muero\n", getpid());

    exit(0);
}
```

## 1.4.7 DESTRUCCIÓN CONTROLADA DEL ÁRBOL.

Tras la ejecución del comando A envía SIGUSR2 a B para indicarle que puede comenzar la destrucción del árbol. B avisa a cada uno de sus hijos en el orden Z, Y y X, mediante kill() y utiliza wait() para asegurarse de que terminan antes de continuar. Una vez finalizados los tres hijos B termina, A termina y por último ejec. De esta manera ningún proceso padre muere antes que sus hijos.

Llamadas al sistema utilizadas: kill(), wait(), exit()

```
    }  
    else{  
        pause();  
  
        kill(pidZ, SIGUSR2);  
        wait(NULL);  
  
        kill(pidY, SIGUSR2);  
        wait(NULL);  
  
        kill(pidX, SIGUSR2);  
        wait(NULL);  
  
        printf("Soy B (%d) y muero\n", getpid());  
  
        exit(0);  
    }  
}
```

```
else{  
    wait(NULL);  
  
    kill(pidB, SIGUSR2);  
    wait(NULL);  
  
    printf("Soy A (%d) y muero\n", getpid());  
  
    exit(0);  
}
```

## EJERCICIO 2

### 2.1 ENUNCIADO DEL EJERCICIO

Realizar un programa llamado hacha.c que divida un archivo en varios trozos con el mismo nombre y extensión h00, h01, .... Se debe tener en cuenta el formato y consideraciones siguientes:

\$ hacha <archivo> <tamaño>

<archivo> nombre del archivo a dividir

<tamaño> tamaño en bytes de los archivos divididos

Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

## 2.1.1 ESTRUCTURA DE PROCESOS

La estructura dependerá del número de fragmentos necesarios para dividir el archivo, por lo que no tenemos una estructura fija como en el ejercicio anterior.

Si que podemos tomar como ejemplo que, si el proceso hacha que actúa como padre tuviera que crear tres hijos porque fueran necesarios tres fragmentos distintos, la estructura se vería tal que así:

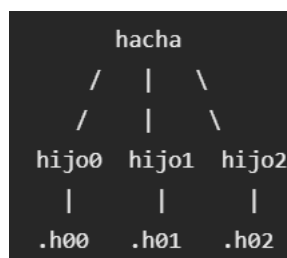
## 2.1.2 RESULTADO ESPERADO

Como indica el enunciado, el resultado esperado en la terminal sería:

```
$ hacha at_madrid.mp3 50000  
$ ls  
at_madrid.mp3.h00  
at_madrid.mp3.h01  
at_madrid.mp3.h02
```

## 2.2 IDENTIFICACIÓN Y JUSTIFICACIÓN DE TAREAS

Según lo que nos pide el enunciado, el problema se puede dividir en varias tareas. La preparación del archivo original, la creación de los procesos y tuberías necesarias, la comunicación de los datos entre los padres e hijos y, finalmente la correcta sincronización y finalización de procesos.



## 2.2.1 RECEPCIÓN DE ARGUMENTOS Y PREPARACIÓN DEL ARCHIVO

El programa recibe dos argumentos, el nombre del archivo a dividir y el tamaño máximo en bytes en el que este se puede dividir.

Tras comprobar que los argumentos son correctos se abrirá el archivo original para su lectura. Teniendo en cuenta el tamaño indicado, el proceso padre irá leyendo la información necesaria para generar cada uno de los fragmentos. Será la llamada al sistema `read()` la que nos permitirá leer un número determinado de bytes de un archivo abierto, puede devolver un número menor de bytes cuando quedan menos datos disponibles.

## 2.2.2 CREACIÓN DE PROCESOS Y TUBERÍAS

Para cada fragmento, el proceso padre ‘hacha’ creará una tubería mediante `pipe()` y, posteriormente, un proceso hijo mediante `fork()`. La tubería la creamos antes que el hijo para que este herede sus descriptores (`fd[0]` y `fd[1]`).

Para crear y gestionar los hijos lo haremos de forma secuencial, de esta manera el padre crea un hijo para cada fragmento, le envía los datos correspondientes y espera a su finalización antes de continuar con el siguiente.

En cada tubería, el padre utilizará el extremo de escritura y el hijo el de lectura.

## 2.2.3 COMUNICACIÓN PADRE E HIJO Y CREACIÓN DE FRAGMENTOS

Tras crear el proceso hijo y la tubería correspondiente, el proceso padre enviará a través de ella los datos al hijo leídos del archivo original. Para ello el padre utiliza el extremo de escritura de la tubería.

El proceso hijo recibe los datos a través del extremo de lectura y será el encargado de crear el archivo destino (.h00, .h01, .h02). Posteriormente, escribe en ese archivo destino los bytes recibidos del padre. Cumpliendo así con la condición principal del enunciado que indica que son los hijos los responsables de la creación y modificación de los archivos destino.

## 2.2.4 SINCRONIZACIÓN Y FINALIZACIÓN

Cuando ya se han enviado los datos correspondientes a un fragmento, el proceso padre cerrará el extremo de escritura de la tubería y ejecutará la llamada al

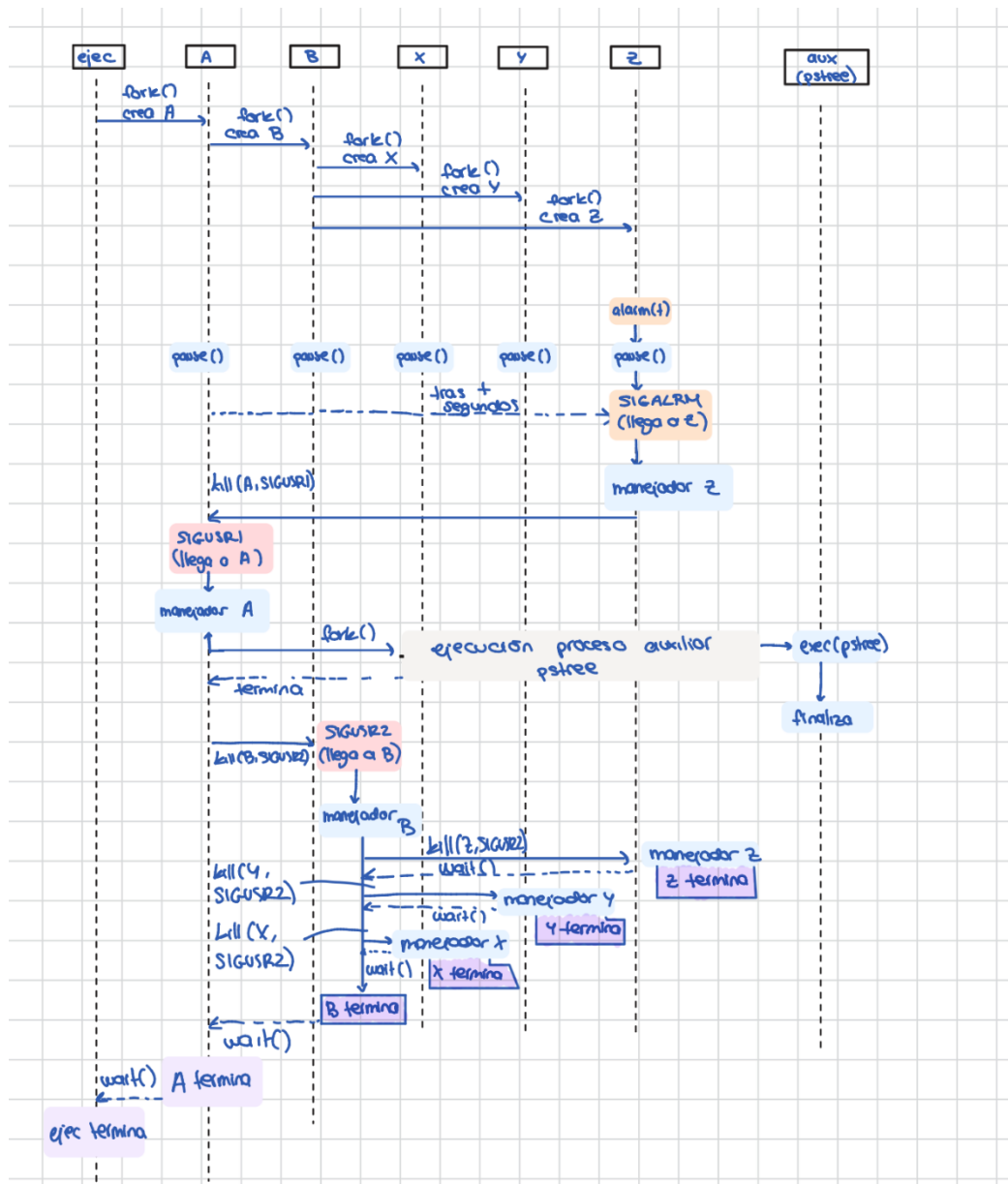
sistema `wait()` para esperar a que el proceso hijo termine de crear y escribir su archivo.

Respecto al hijo, este cerrará los descriptores que ya no necesite una vez finalizadas las operaciones de lectura y escritura. Al realizar los procesos de forma secuencial, el padre no continuará con el fragmento siguiente hasta que el hijo anterior no haya finalizado.

Este procedimiento se repetirá hasta procesar por completo el archivo original. Finalmente, se cerrarán los descriptores abiertos y el proceso `hacha` terminará su ejecución.

## 2.3 DIAGRAMA DE SECUENCIA

El siguiente diagrama de secuencia muestra la interacción entre el proceso padre `'hacha'` y los procesos hijos durante la creación de un fragmento. Este proceso se repite para cada uno de los fragmentos del archivo, en este caso el ejemplo se ha realizado creando dos hijos, pero podrían ser más.



## 2.4 ANÁLISIS DETALLADO DEL CÓDIGO FUENTE

En este apartado se procede a analizar el código realizado para desarrollar el ejercicio, diferenciando las tareas responsabilidad del proceso hacha y las de los procesos hijos.

### 2.4.1 FUNCION VALIDAR\_argumentos()

Esta función se encarga de comprobar que el programa haya recibido correctamente los dos argumentos requeridos, el nombre del archivo que se desea dividir y el tamaño de cada fragmento.

También convierte el tamaño recibido a un valor entero con la función `atoi()` y comprueba que sea mayor que cero. Si alguno de los requisitos no se cumple, el programa finaliza indicando error. De lo contrario, la función devuelve el tamaño que será utilizado durante la división del archivo.

Llamadas al sistema utilizadas: `exit()`

```
int validar_argumentos(int argc, char *argv[]){
    int tam;

    if (argc != 3) {
        printf("Uso: hacha <archivo> <tamano>\n");
        exit(1);
    }

    tam = atoi(argv[2]);

    if (tam <= 0) {
        printf("El tamaño debe ser mayor que 0\n");
        exit(1);
    }

    return tam;
}
```

## 2.4.2 FUNCION Main()

En esta función ocurre la coordinación principal del programa. Primero llama a `validar_argumentos()` y abre el archivo original únicamente para lectura mediante `open()`.

A continuación reserva un búfer del tamaño indicado y utiliza `read()` para leer sucesivamente los fragmentos del archivo. Por cada bloque leído se llama a `procesar_fragmento()`, que se encargará de crear la tubería y el proceso hijo correspondiente.

Continuará hasta que `read()` devuelva cero, indicando así que se ha llegado al final del archivo. Por último, se cierra el descriptor del archivo original.

Llamadas al sistema utilizadas: `open()`, `read()`, `close()`, `exit()`

```

int main(int argc, char *argv[]) {
    int fdArchivo, tam, bytesLeidos;
    int contador = 0;

    tam = validar_argumentos(argc, argv);

    fdArchivo = open(argv[1], O_RDONLY);

    if (fdArchivo == -1){
        perror("Error al abrir el archivo");
        exit(1);
    }

    char buffer[tam];

    while ((bytesLeidos = read(fdArchivo, buffer, tam)) > 0){
        procesar_fragmento(buffer, bytesLeidos, argv[1], contador);
        contador++;
    }

    if (bytesLeidos == -1){
        perror("Error al leer el archivo");
        close(fdArchivo);
        exit(1);
    }

    close(fdArchivo);
    return 0;
}

```

## 2.4.3 FUNCION PROCESAR\_FRAGMENTO()

La función `procesar_fragmento()` se encarga de gestionar la creación y comunicación con el proceso hijo correspondiente a cada fragmento. Primero se crea la tubería mediante `pipe()` y, a continuación, se genera el proceso hijo utilizando la llamada al sistema `fork()`.

El proceso hijo cierra el extremo de escritura de la tubería y llama a `proceso_hijo()` para recibir y almacenar el fragmento. Por otro lado, el proceso padre cierra el extremo de lectura y envía los datos mediante `write()`.

Para terminar, el padre cierra la tubería y utiliza `wait()` para esperar a que el hijo termine antes de continuar con el siguiente fragmento. De esta forma, los hijos se ejecutan secuencialmente tal y como habíamos representado en el diagrama de secuencia.

Llamadas al sistema utilizadas: `pipe()`, `fork()`, `write()`, `close()`, `wait()`, `exit()`.

```

void procesar_fragmento(char *buffer, int bytesLeidos, char *nombreArchivo, int numeroFragmento) {
    int tuberia[2];
    pid_t pid;

    if (pipe(tuberia) == -1){
        perror("Error al crear la tuberia");
        exit(1);
    }

    pid = fork();

    if (pid == -1){
        perror("Error en fork");
        close(tuberia[0]);
        close(tuberia[1]);
        exit(1);
    }

    if (pid == 0){ //hijo
        close(tuberia[1]);

        proceso_hijo(tuberia[0], nombreArchivo, numeroFragmento, bytesLeidos);

        close(tuberia[0]);
        exit(0);
    }
}

```

```

else { //padre
    close(tuberia[0]);

    if (write(tuberia[1], buffer, bytesLeidos) == -1){
        perror("Error al escribir en la tuberia");
        close(tuberia[1]);
        exit(1);
    }

    close(tuberia[1]);
    wait(NULL);
}
}

```

## 2.4.4 FUNCION PROCESO\_HIJO()

Esta función `proceso_hijo()` se encarga de recibir los datos enviados por el padre a través de la tubería y crear el archivo correspondiente a cada fragmento.

El hijo lee de la tubería mediante `read()` hasta recibir todos los bytes que forman su fragmento. Después construye el nombre del archivo de salida añadiendo la extensión correspondiente (`.h00`, `.h01`, etc) y crea dicho archivo mediante `creat()`.

Por último, escribe en el archivo creado los datos recibidos usando `write()`, cierra el descriptor `close()`. De esta forma se cumple la condición del enunciado de que sean los procesos hijos quienes creen y escriban los archivos destino.

Llamadas al sistema utilizadas: `read()`, `creat()`, `write()`, `close()`, `exit()`

```

void proceso_hijo(int fdLectura, char *nombreArchivo, int numeroFragmento, int tam){
    char datos[tam];
    char nombreDestino[256];
    int fdDestino, n;
    int totalLeido = 0;

    while (totalLeido < tam){
        n = read(fdLectura, datos + totalLeido, tam - totalLeido);

        if (n == -1) {
            perror("Error al leer de la tuberia");
            exit(1);
        }

        if (n == 0) {
            printf("Error al recibir el fragmento\n");
            exit(1);
        }

        totalLeido = totalLeido + n;
    }
}

```

```

sprintf(nombreDestino, "%s.h%02d", nombreArchivo, numeroFragmento);

fdDestino = creat(nombreDestino, 0644);

if (fdDestino == -1){
    perror("Error al crear el archivo");
    exit(1);
}

if (write(fdDestino, datos, totalLeido) == -1){
    perror("Error al escribir en el archivo");
    close(fdDestino);
    exit(1);
}

close(fdDestino);

```

## 2.4.5 Ejecución

```

alumno@alumno-VirtualBox:~/S0/Practical/Ejercicio2$ echo "Este es un archivo de
prueba para comprobar el funcionamiento del programa hacha. Vamos a poner bastan
te texto para poder dividirlo en varios fragmentos." > prueba.txt
alumno@alumno-VirtualBox:~/S0/Practical/Ejercicio2$ ls
hacha hacha.c prueba.txt
alumno@alumno-VirtualBox:~/S0/Practical/Ejercicio2$ ./hacha prueba.txt 50
alumno@alumno-VirtualBox:~/S0/Practical/Ejercicio2$ ls -l
total 44
-rwxrwxr-x 1 alumno alumno 16632 oct  6 16:50 hacha
-rw-rw-r-- 1 alumno alumno  2796 oct  6 16:49 hacha.c
-rw-rw-r-- 1 alumno alumno   154 oct  6 16:52 prueba.txt
-rw-r--r-- 1 alumno alumno    50 oct  6 16:52 prueba.txt.h00
-rw-r--r-- 1 alumno alumno    50 oct  6 16:52 prueba.txt.h01
-rw-r--r-- 1 alumno alumno    50 oct  6 16:52 prueba.txt.h02
-rw-r--r-- 1 alumno alumno     4 oct  6 16:52 prueba.txt.h03

```



## 3.2 IDENTIFICACIÓN Y JUSTIFICACIÓN DE TAREAS

El programa generará una estructura formada por una cadena vertical de  $x$  procesos  $y$ , desde el último proceso se deberán de crear  $y$  procesos finales. El proceso inicial debe de conocer el PID de todos los procesos finales y estos conocerán el PID de todos sus antecesores. Para poder compartir esta información entre procesos se utilizará la memoria compartida, que es la novedad que se introduce en este ejercicio.

### 3.2.1 RECEPCIÓN Y VALIDACIÓN DE ARGUMENTOS

El programa recibe dos argumentos,  $x$  que indica el numero de procesos que formarán la cadena vertical  $y$ , por otro lado,  $y$ , que determina la cantidad de proceso finales que creará el último proceso de dicha cadena vertical.

Será necesario realizar dos comprobaciones para el correcto desarrollo del programa. En primer lugar, que hayan exactamente dos argumentos  $y$ , a continuación, que estos valores sean válidos, es decir, mayores que cero.

### 3.2.2 CREACIÓN DE LA CADENA DE $x$ PROCESOS

El proceso inicial  $hijos$  debe ir creando la cadena vertical mediante `fork()`. Cada proceso de la cadena crea únicamente al siguiente, esto lo conseguimos identificando al hijo del padre tras la llamada al sistema para asegurarnos de que se cumple la estructura de padre e hijo sucesivamente.

Además, es necesario que durante esta creación guardemos los identificadores de los procesos, ya que deberán de ser conocidos por los procesos finales.

### 3.2.3 CREACIÓN DE LOS $y$ PROCESOS FINALES

Cuando llegamos al último proceso de la cadena vertical, este será el responsable de crear los  $y$  procesos finales.

A diferencia de la dinámica utilizada para la cadena vertical, todos estos procesos tendrán el mismo padre, que como ya hemos explicado es el ultimo proceso de la cadena.

Para poder mostrar por terminal el mensaje que nos indica el enunciado es importante que estos procesos conozcan tanto su propio identificador como el de todos sus antecesores.

### 3.2.4 COMUNICACIÓN MEDIANTE MEMORIA COMPARTIDA

Como ya hemos mencionado, en este ejercicio la novedad es precisamente la necesidad de comunicación entre procesos para intercambiar información, en concreto los identificadores de los distintos procesos, tanto por parte del proceso principal que necesita saber todos los PID de sus procesos hijos hasta los finales y, por parte de los y procesos finales que han de saber el PID de los antecesores.

Para que la información pueda ser usada por los distintos procesos empleamos la memoria compartida, una zona de memoria común en la que almacenar y consultar los identificadores.

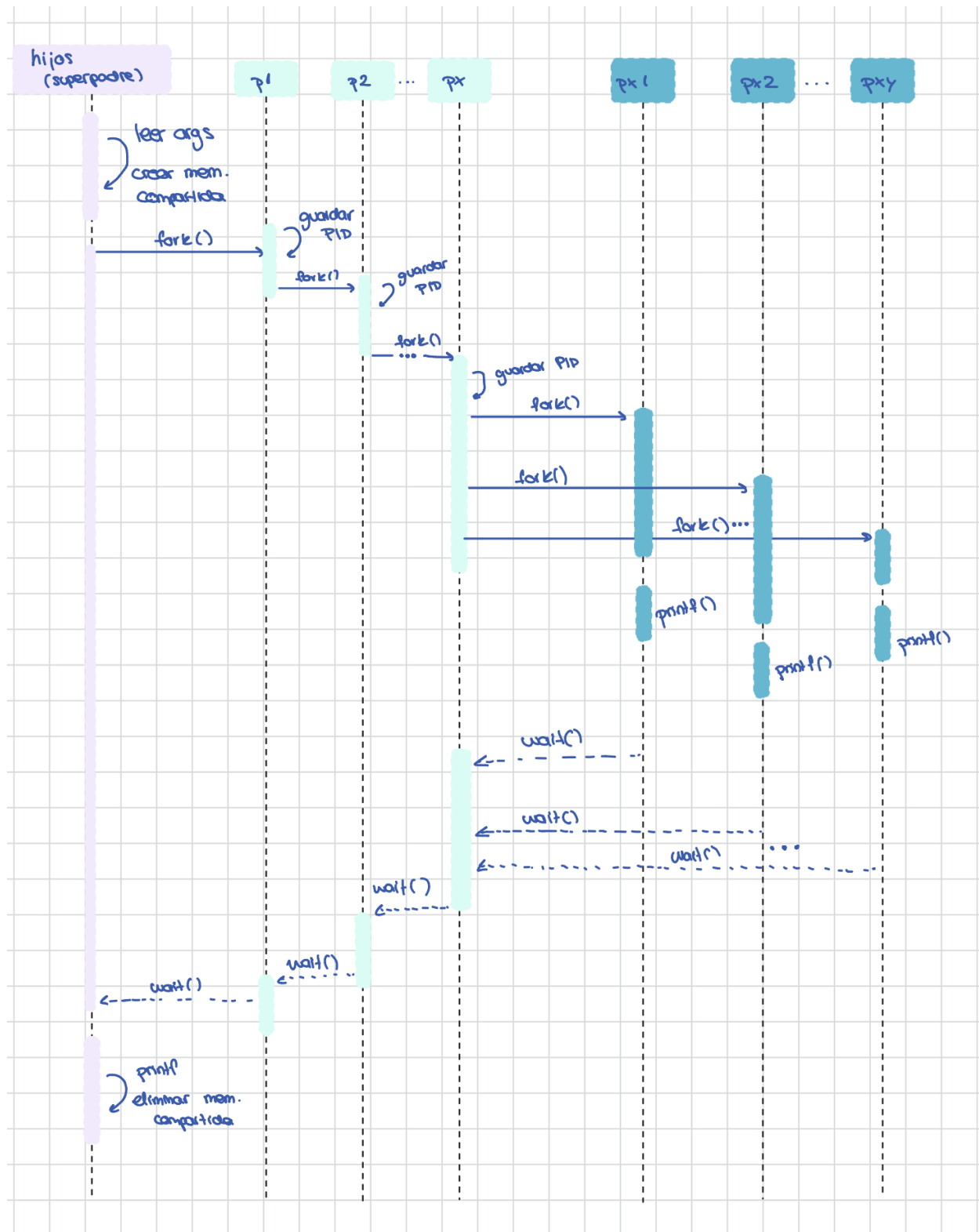
### 3.2.5 SINCRONIZACIÓN Y FINALIZACIÓN DE LOS PROCESOS

La ejecución de los procesos ha de estar coordinada para que garantizar que la información necesaria ha sido almacenada antes de utilizarla.

Los procesos de la cadena deben permanecer activos mientras se crean el resto de los procesos situados por debajo. Al igual que el ultimo proceso de la cadena deberá de esperar a que finalicen todos los procesos finales que ha generado.

La finalización se realizará de forma ordenada, desde la parte inferior del árbol hacia arriba, evitando así que los procesos padres terminen antes que los hijos.

## 3.3 DIAGRAMA DE SECUENCIA



## 3.4 ANÁLISIS DETALLADO DEL CÓDIGO FUENTE

A continuación, se realiza un análisis del código utilizado para desarrollar el ejercicio.

### 3.4.1 FUNCIÓN `VALIDAR_ARGUMENTOS()`

Esta función se encarga de comprobar que el programa haya recibido correctamente los dos argumentos necesarios para el programa.

Los valores recibidos se convierten en enteros mediante `atoi()` y se comprueba que ambos sean mayores que cero. Previamente se habrá comprobado que se haya introducido el número correcto de argumentos. Si alguno de los requisitos no se cumple, el programa muestra un mensaje de error y finaliza.

Llamadas al sistema utilizadas: `exit()`.

```
void validar_argumentos(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Error: número de argumentos inválido\n");
        exit(1);
    }

    if (atoi(argv[1]) <= 0 || atoi(argv[2]) <= 0) {
        printf("Los valores de x e y deben ser mayores que 0\n");
        exit(1);
    }
}
```

### 3.4.2 FUNCIÓN `MAIN()`

La función `main()` se encarga de coordinar el funcionamiento general del programa. Primero llama a `validar_argumentos()` y crea un segmento de memoria compartida mediante `shmget()`, con espacio suficiente para almacenar los identificadores necesarios.

A continuación, vincula la memoria compartida mediante `shmat()` y crea el primer proceso hijo utilizando `fork()`. Este hijo almacena su PID y llama a `crear_cadena()` para continuar construyendo el árbol, mientras que el proceso inicial espera mediante `wait()`.

Una vez finalizados todos los procesos descendientes, el superpadre llama a `mostrar_superpadre()` para mostrar los identificadores de los hijos finales. Para terminar, se desvincula la memoria compartida mediante `shmdt()` y elimina el segmento utilizando `shmctl()`.

Llamadas al sistema utilizadas: `shmget()`, `shmat()`, `fork()`, `getpid()`, `wait()`, `shmdt()`, `shmctl()`, `exit()`.

```

int main(int argc, char *argv[]) {
    int x, y;
    int idMemoria;
    pid_t *memoria;
    pid_t pid;

    validar_argumentos(argc, argv);

    x = atoi(argv[1]);
    y = atoi(argv[2]);

    idMemoria = shmget(IPC_PRIVATE, (x + y) * sizeof(pid_t), IPC_CREAT | 0600);

    if (idMemoria == -1) {
        perror("Error al crear memoria compartida");
        exit(1);
    }

    memoria = (pid_t *) shmat(idMemoria, NULL, 0);

    if (memoria == (void *) -1) {
        perror("Error al vincular memoria compartida");
        shmctl(idMemoria, IPC_RMID, NULL);
        exit(1);
    }
}

```

```

pid = fork();

if (pid == -1) {
    perror("Error en fork");
    shmdt(memoria);
    shmctl(idMemoria, IPC_RMID, NULL);
    exit(1);
}

if (pid == 0) {
    memoria[0] = getpid();
    crear_cadena(x, y, 1, memoria);
    shmdt(memoria);
    exit(0);
}
else {
    wait(NULL);
    mostrar_superpadre(x, y, memoria);
    shmdt(memoria);
    shmctl(idMemoria, IPC_RMID, NULL);
}

return 0;
}

```

### 3.4.3 Función crear\_cadena()

Esta función continuará la construcción de la cadena vertical de procesos, una vez que el proceso inicial ha creado el primer hijo.

Cada proceso crea al siguiente mediante `fork()` y almacena su identificador en la posición correspondiente de la memoria compartida. Esto se repite hasta alcanzar los `x` procesos.

Cuando se completa la cadena, el último proceso llama a `crear_hijos_finales()` para generar los procesos de la parte inferior del árbol. Los procesos padres utilizan `wait()` para esperar a que terminen sus hijos y realizar así una finalización ordenada.

Llamadas al sistema utilizadas: `fork()`, `getpid()`, `wait()`, `shmdt()`, `exit()`.

```

void crear_cadena(int x, int y, int posicion, pid_t *memoria) {
    pid_t pid;

    if (posicion == x) {
        crear_hijos_finales(x, y, memoria);
    }
    else {
        pid = fork();

        if (pid == -1) {
            perror("Error en fork");
            exit(1);
        }

        if (pid == 0) {
            memoria[posicion] = getpid();
            crear_cadena(x, y, posicion + 1, memoria);
            shmdt(memoria);
            exit(0);
        }
        else {
            wait(NULL);
        }
    }
}

```

### 3.4.4 FUNCIÓN crear\_HIJOS\_FINALES()

Esta función la ejecuta el último proceso de la cadena y se encarga de generar los y procesos finales mediante fork().

Cada proceso hijo muestra su propio identificador mediante getpid() y accede a la memoria compartida para consultar los identificadores de sus antecesores, mostrando el mensaje que indica el enunciado.

El proceso padre almacena en la memoria compartida los PID de los hijos finales que va creando. Una vez generados todos, utiliza wait() para esperar a que terminen.

Llamadas al sistema utilizadas: fork(), getpid(), shmdt(), exit(), wait().

```

void crear_hijos_finales(int x, int y, pid_t *memoria) {
    int i, j;
    pid_t pid;

    for (i = 0; i < y; i++) {
        pid = fork();

        if (pid == -1) {
            perror("Error en fork");
            exit(1);
        }

        if (pid == 0) {
            printf("Soy el subhijo %d, mis padres son: ", getpid());

            for (j = 0; j < x; j++) {
                printf("%d", memoria[j]);

                if (j < x - 1) {
                    printf(", ");
                }
            }

            printf("\n");
            shmdt(memoria);
            exit(0);
        }
        else {
            memoria[x + i] = pid;
        }
    }

    for (i = 0; i < y; i++) {
        wait(NULL);
    }
}

```

### 3.4.5 FUNCIÓN MOSTRAR\_SUPERPADRE()

Esta función se dedica a mostrar el mensaje correspondiente al proceso inicial hijos.

Una vez finalizada la ejecución de los procesos descendientes, el superpadre consulta la memoria compartida para obtener los identificadores de los y procesos finales y mostrarlos por pantalla junto con su propio PID.

Llamadas al sistema utilizadas: getpid()

```
void mostrar_superpadre(int x, int y, pid_t *memoria) {
    int i;

    printf("Soy el superpadre (%d): mis hijos finales son: ", getpid());

    for (i = 0; i < y; i++) {
        printf("%d", memoria[x + i]);

        if (i < y - 1) {
            printf(", ");
        }
    }

    printf("\n");
}
```

### 3.4.6 EJECUCIÓN

```
alumno@alumno-VirtualBox:~/S0/Practical/Ejercicio3$ gcc -Wall -o hijos hijos.c
alumno@alumno-VirtualBox:~/S0/Practical/Ejercicio3$ ./hijos 3 4
Soy el subhijo 4240, mis padres son: 4237, 4238, 4239
Soy el subhijo 4241, mis padres son: 4237, 4238, 4239
Soy el subhijo 4242, mis padres son: 4237, 4238, 4239
Soy el subhijo 4243, mis padres son: 4237, 4238, 4239
Soy el superpadre (4236): mis hijos finales son: 4240, 4241, 4242, 4243
alumno@alumno-VirtualBox:~/S0/Practical/Ejercicio3$
```